

Contest Log Analytics - Programmer's Guide

Version: 1.0.0-alpha.18 **Date:** 2026-01-24 **Last Updated:** 2026-01-31 **Category:** Programmer Reference

--- Revision History ---

[1.0.0-alpha.18] - 2026-01-31

Changed

- Updated version to match project release v1.0.0-alpha.18 (no content changes)

[1.0.0-alpha.17] - 2026-01-30

Changed

- Updated version to match project release v1.0.0-alpha.17 (no content changes)

[1.0.0-alpha.16] - 2026-01-30

Changed

- Updated version to match project release v1.0.0-alpha.16 (no content changes)

[1.0.0-alpha.15] - 2026-01-30

Changed

- Updated version to match project release v1.0.0-alpha.15 (no content changes)

[1.0.0-alpha.14] - 2026-01-26

Changed

- Updated version to match project release v1.0.0-alpha.14 (no content changes)

[1.0.0-alpha.13] - 2026-01-26

Changed

- Updated version to match project release v1.0.0-alpha.13 (no content changes)

[1.0.0-alpha.11] - 2026-01-24

Changed

- Updated version to match project release v1.0.0-alpha.11 (no content changes)

[1.0.0-alpha.10] - 2026-01-24

Changed

- Updated version to match project release v1.0.0-alpha.10 (no content changes)

1. Introduction & Architecture

The Contest Log Analytics project is built on three core architectural principles:

- **Data-Driven:** The behavior of the analysis engine is primarily controlled by data, not code. Contest rules, multiplier definitions, and parsing logic are defined in simple `.json` files.
- **Extensible:** The application uses a "plugin" architecture. New reports and contest-specific logic modules are dynamically discovered at runtime.
- **Convention over Configuration:** Files and classes must be named and placed in specific, predictable locations to be discovered.

Documentation Index

Purpose: Central reference for all maintained documentation. See `Docs/VersionManagement.md` Section 3 for complete versioning policy.

User-Facing Documentation (Category A - Match Project Version)

- `UsersGuide.md` - Version: 1.0.0-alpha.11
- `InstallationGuide.md` - Version: 1.0.0-alpha.11
- `ReportInterpretationGuide.md` - Version: 1.0.0-alpha.11

Programmer Reference (Category B - Match Project Version)

- `ProgrammersGuide.md` - Version: 1.0.0-alpha.11
- `PerformanceProfilingGuide.md` - Version: 1.0.0-alpha.11

Algorithm Specifications (Category C - Independent Versioning)

- `CallsignLookupAlgorithm.md` - Version: 0.90.11-Beta (Compatible with: up to v1.0.0-alpha.11)

- `RunS&PAlgorithm.md` - Version: 0.47.4-Beta (Compatible with: up to v1.0.0-alpha.11)
- `WPXPrefixLookup.md` - Version: 0.70.0-Beta (Compatible with: up to v1.0.0-alpha.11)

Style Guides (Category D - Independent Versioning)

- `CLAReportsStyleGuide.md` - Version: 1.4.0

For versioning policy details, see: `Docs/VersionManagement.md` Section 3

2. The Data Aggregation Layer (DAL) - CRITICAL ARCHITECTURAL PATTERN

THE DAL IS THE SOLE AUTHORITY FOR DATA PROCESSING.

Reports **MUST NOT** process raw log data directly.

Core Principle: Separation of Concerns

- **Data Processing:** Performed exclusively by aggregators in `contest_tools.data_aggregators`
- **Report Generation:** Performed by reports in `contest_tools.reports`, which consume aggregator output
- **Time Alignment:** Aggregators ensure consistent time bin alignment across all logs using `master_time_index`
- **Data Format:** All Aggregators return standard Python primitives (Dictionaries, Lists, Ints), never Pandas DataFrames

CRITICAL RULES FOR REPORT IMPLEMENTATION

1. **DO NOT process raw log data in reports.** Use aggregators instead.
2. **DO NOT create pivot tables, groupby operations, or time binning in reports.** This belongs in aggregators.
3. **DO NOT manually iterate over logs to extract data.** Use aggregator methods that handle all logs consistently.
4. **DO use `master_time_index` from `LogManager`** to ensure time alignment when passing `time_index` to aggregators.
5. **DO use existing aggregator methods** or extend aggregators with new methods for new data structures.

Why This Matters

- **Time Alignment:** Manual time binning in reports can cause misalignment across logs, leading to rendering bugs where only some logs appear correctly.
- **Consistency:** Aggregators ensure all logs are processed identically, preventing data structure mismatches.

- **Maintainability:** Centralized data processing logic is easier to test, debug, and extend.
- **Performance:** Aggregators can be cached and reused across multiple reports.

Primary Aggregators

- **CategoricalAggregator:** Handles set operations (Unique/Common QSOs) and categorical grouping.
- **ComparativeEngine:** Implements Set Theory logic to calculate Universe, Common, Differential, and Missed counts.
- **MatrixAggregator:**
 - `get_stacked_matrix_data()`: Generates 3D data (Band x Time x RunStatus) for stacked charts
 - `get_mode_stacked_matrix_data()`: Generates 3D data (Mode x Time x RunStatus) for mode-dimension charts
 - `get_matrix_data()`: Generates 2D grids (Band x Time) for heatmaps
- **MultiplierStatsAggregator:** Handles "Missed Multiplier" analysis and summarization.
 - `get_multiplier_breakdown_data(dimension='band'|'mode')`: Generates hierarchical multiplier breakdown by band or mode dimension (automatically selects mode dimension for single-band, multi-mode contests).
 - `get_missed_data(mult_name, mode_filter=None, enhanced=False)`: Returns missed multiplier analysis. When `enhanced=True` (Sweepstakes only), includes detailed breakdown with worked-by callsigns, bands/modes, and Run/S&P/Unknown counts.
- **TimeSeriesAggregator:** Generates the standard TimeSeries Data Schema (v1.4.0).
- **WaeStatsAggregator:** Specialized logic for WAE contests (QTCs and weighted multipliers).

Example: Correct Pattern (Using DAL)

```
# CORRECT: Report uses aggregator
def generate(self, output_path: str, **kwargs):
    # Get master_time_index for alignment
    master_index = None
    if self.logs and hasattr(self.logs[0], '_log_manager_ref'):
        log_manager = self.logs[0]._log_manager_ref
        if log_manager:
            master_index = log_manager.master_time_index

    # Use aggregator - do NOT process raw data
    matrix_agg = MatrixAggregator(self.logs)
```

```
matrix_data = matrix_agg.get_stacked_matrix_data(bin_size='60min', time_index=master_index)

# Report only formats/visualizes the pre-processed data
# ... visualization code ...
```

Example: Anti-Pattern (Manual Processing)

```
# WRONG: Report manually processes data
def generate(self, output_path: str, **kwargs):
    for log in self.logs:
        df = get_valid_dataframe(log)
        # BAD: Manual pivot tables, time binning, etc. in report
        pivot = df.pivot_table(...)
        # This causes time misalignment and inconsistent data structures!
```

3. Web Architecture (Phase 3)

The project includes a stateless, containerized web dashboard built on Django.

The Manifest & WORM Strategy

To ensure responsiveness, the application employs a **”Write Once, Read Many” (WORM)** strategy.

1. **Generation Phase:** During analysis, ”Artifact Reports” (e.g., `json_multiplier_breakdown.py`) execute and serialize aggregation trees into JSON files.
2. **Hydration Phase:** When a user loads a dashboard view, the view hydrates its context directly from these pre-computed JSON artifacts via the `ManifestManager`.
3. **Result:** No re-parsing of Cabrillo logs occurs on page load.

Dashboard Chart Embedding Architecture

The dashboard uses two distinct approaches for displaying charts, each optimized for different use cases:

Iframe Embedding (Legacy - Text Reports) Use Case: Static HTML reports, text-based reports, or charts with fixed dimensions.

Implementation:

- Charts are saved as standalone HTML files
- Embedded via nested iframes: `qso_dashboard.html` & `report_viewer.html` HTML file
- Fixed-height iframes provide consistent viewport sizing

- Works well for reports that don't require dynamic resizing

Limitations:

- Fixed-height iframes constrain content and prevent true `autosize=True` behavior
- Title annotations with fixed pixel offsets (`yshift`) don't scale proportionally in constrained iframes
- Double iframe nesting adds complexity and can cause layout issues

Direct Plotly Embedding (Modern - Interactive Charts) Use Case: Interactive Plotly charts with `autosize=True`, subplot charts, or charts requiring responsive behavior.

Implementation:

- Charts are saved as both HTML (for full-screen viewing) and JSON (for embedding)
- JSON files are loaded via JavaScript and rendered directly into `<div>` elements using `Plotly.newPlot()`
- Charts can truly autosize to fill their container
- `ResizeObserver` handles dynamic resizing

Benefits:

- True responsive behavior - charts adapt to container size
- No iframe constraints - title positioning works correctly
- Better performance - direct DOM manipulation
- Proper legend positioning - no overlap with titles

When to Use Each Approach:

- **Use Direct Embedding:** For all new Plotly charts, especially subplot charts or charts with multi-line titles
- **Use Iframe Embedding:** For text reports, static HTML, or legacy charts that haven't been migrated yet

Example - Direct Embedding Pattern:

```
// Load JSON and render directly in div
fetch(jsonUrl)
  .then(response => response.json())
  .then(data => {
    Plotly.newPlot('chart-container', data.data, data.layout, {
      responsive: true,
      displayModeBar: false
    });
  });
```

Migration Note: The "Unique QSO Band Distribution" chart was migrated from iframe to direct embedding to resolve title overlap and legend positioning

issues in the dashboard. This pattern should be used for all future Plotly chart implementations.

4. Shared Utilities & Styles

- **MPLStyleManager**: Centralized Matplotlib styles for static plots.
 - **PlotlyStyleManager**: Centralized Plotly styles for interactive visualizations.
 - `get_point_color_map()`: Standard color mapping for QSO points.
 - `get_qso_mode_colors()`: Standard scheme for Run/S&P modes.
 - **CtyManager**: Manages the lifecycle and caching of the `cty.dat` country file.
-

5. How to Add a New Contest

To add a new contest (e.g., "My Contest"), you generally do not need to write Python code. You only need to define the rules in JSON.

Step 1: Create the Definition File

Create a JSON file in `contest_tools/contest_definitions/` with a filename that matches your contest name:

- Convert the contest name to lowercase
- Replace hyphens (-) and spaces with underscores (_)
- Add `.json` extension

Example: For contest name MY-CONTEST, create `my_contest.json`

Important: The `contest_name` field in the JSON must **exactly match** the `CONTEST:` tag in the Cabrillo file header (case-sensitive).

Step 2: Define Basic Metadata

Every contest definition requires these core fields:

```
{
  "contest_name": "MY-CONTEST",
  "dupe_check_scope": "per_band",
  "score_formula": "points_times_mults",
  "valid_bands": ["160M", "80M", "40M", "20M", "15M", "10M"]
}
```

Field Descriptions:

- `contest_name`: Must exactly match the `CONTEST:` header in Cabrillo files

- `dupe_check_scope`: Either `"per_band"` (duplicates checked per band) or `"all_bands"` (duplicates checked across all bands)
- `score_formula`: Scoring calculation method:
 - `"points_times_mults"`: Standard formula (QSO points × multipliers)
 - `"qsos_times_mults"`: QSO count × multipliers (e.g., NAQP)
 - `"total_points"`: Sum of points only (e.g., ARRL Field Day)
 - `"custom"`: Requires a `scoring_module` (e.g., WAE contests)
- `valid_bands`: Array of allowed band names (standard: 160M, 80M, 40M, 20M, 15M, 10M, 6M, 2M, etc.)

Step 3: Define Exchange Parsing Rules

Most contests require `exchange_parsing_rules` to parse QSO exchanges from Cabrillo format. This defines regex patterns to extract exchange components.

```
"exchange_parsing_rules": {
  "MY-CONTEST": {
    "regex": "(\\d{3})\\s+([A-Z]{2})\\s+([A-Z0-9/]+)\\s+(\\d{3})\\s+([A-Z]{2})",
    "groups": [
      "SentRST",
      "SentLocation",
      "Call",
      "RST",
      "RcvdLocation"
    ]
  }
}
```

Notes:

- The key (e.g., `"MY-CONTEST"`) must match the contest name from the Cabrillo header
- The `regex` field uses standard regex syntax with capturing groups
- The `groups` array maps each capturing group to a column name
- For contests with different exchange formats (e.g., W/VE vs DX), define multiple keys (see `arrl_dx_cw.json` for examples)

Step 4: Define Multiplier Rules

Multiplier rules define how multipliers are calculated and counted. **This must be an array, not an object.**

```
"multiplier_rules": [
  {
    "name": "Sections",
    "value_column": "Multi",
    "name_column": "MultiName",
  }
]
```



```

        "totaling_method": "once_per_log"
    },
    {
        "name": "Zones",
        "source_column": "Zone",
        "value_column": "Mult2",
        "totaling_method": "sum_by_band"
    }
]

```

Common Field Descriptions:

- **name:** Display name for the multiplier type
- **value_column:** DataFrame column storing multiplier values (typically Mult1, Mult2, etc.)
- **name_column:** DataFrame column storing multiplier names (typically Mult1Name, Mult2Name, etc.)
- **source_column:** Source column to extract multiplier from (e.g., Zone from QSO data)
- **source:** Pre-defined multiplier source (e.g., "wae_dxcc" for WAE DXCC list)
- **totaling_method:** How multipliers are counted:
 - "once_per_log": Count each multiplier once across entire log
 - "sum_by_band": Sum multipliers across all bands (same multiplier on different bands counts separately)
 - "once_per_mode": Count once per mode (for multi-mode contests)
- **applies_to:** Optional filter (e.g., "W/VE" or "DX" for location-specific multipliers)

Step 5: Define Default QSO Columns (Optional)

Specify which columns appear in reports:

```

"default_qso_columns": [
    "ContestName",
    "MyCall",
    "Frequency",
    "Mode",
    "Datetime",
    "SentRST",
    "SentLocation",
    "Call",
    "RST",
    "RcvdLocation",
    "Band",
    "Date",
    "Hour",

```

```

    "Dupe",
    "DXCCName",
    "DXCCPfx",
    "Run",
    "QSOPoints",
    "Mult1",
    "Mult1Name"
]

```

If omitted, the system uses defaults from `_common_cabrillo_fields.json`.

Step 6: Additional Optional Fields

Many contests require additional configuration:

Scoring & Calculation:

- `scoring_module`: Python module name for custom scoring (e.g., `"wae_scoring"`)
- `time_series_calculator`: Custom time series calculation module (e.g., `"naqp_calculator"`)

Custom Parsers & Resolvers:

- `custom_parser_module`: Custom Cabrillo parser module (e.g., `"arrl_dx_parser"`)
- `custom_multiplier_resolver`: Custom multiplier resolution logic (e.g., `"arrl_ss_multiplier_resolver"`)
- `custom_location_resolver`: Custom location determination (e.g., `"wae_location_resolver"`)
- `custom_dupe_checker`: Custom duplicate QSO checking logic (e.g., `"laqp_dupe_checker"`)
- `custom_adif_exporter`: Custom ADIF export logic (e.g., `"cq_ww_adif"`)

Contest Period & Operating Rules:

- `contest_period`: Defines contest start/end times:


```

"contest_period": {
    "start_day": "Saturday",
    "start_time": "00:00:00 UTC",
    "end_day": "Sunday",
    "end_time": "23:59:59 UTC"
}

```
- `operating_time_rules`: Operating time restrictions:


```

"operating_time_rules": {
    "min_off_time_minutes": 30,
    "single_op_max_hours": 24,
    "multi_op_max_hours": 24
}

```

Report Control:

- **excluded_reports:** Array of report names to skip (e.g., ["text_wae_score_report"])
- **included_reports:** Array of report names to include (if specified, only these run)

Multiplier Behavior:

- **mults_from_zero_point_qsos:** Boolean - whether zero-point QSOs can yield multipliers (default: `true`)
- **mutually_exclusive_mults:** Array of arrays defining mutually exclusive multiplier groups:

```
"mutually_exclusive_mults": [  
  ["Mult_STPROV", "Mult_NADXCC"]  
]
```
- **multiplier_report_scope:** Scope for multiplier reports ("per_band" or "all_bands")

Other:

- **enable_adif_export:** Boolean - enable ADIF export functionality
- **points_header_label:** Custom label for points column in reports (e.g., "QSO+QTC Pts" for WAE)
- **is_naqp_ruleset:** Boolean - special flag for NAQP contest rules

Inheritance Mechanisms

The system supports two inheritance mechanisms:

1. Explicit Inheritance (`inherits_from`):

```
{  
  "inherits_from": "cq_ww_cw",  
  "contest_name": "MY-CONTEST",  
  "score_formula": "points_times_mults"  
}
```

The child definition inherits all fields from the parent, then overrides with its own values.

2. Implicit Generic/Specific Pattern: If a specific contest file isn't found (e.g., `arrl_ss_cw.json`), the system automatically tries the generic version (e.g., `arrl_ss.json`) by removing the last hyphen-separated segment.

3. Common Fields Base: All contest definitions automatically inherit from `_common_cabrillo_fields.json`, which provides:

- Standard Cabrillo header field mappings
- Default QSO column definitions
- Common field structures

Complete Example

Here's a complete example for a simple contest:

```
{
  "_filename": "contest_tools/contest_definitions/my_contest.json",
  "contest_name": "MY-CONTEST",
  "scoring_module": "my_contest_scoring",
  "valid_bands": ["80M", "40M", "20M", "15M", "10M"],
  "dupe_check_scope": "per_band",
  "score_formula": "points_times_mults",
  "enable_adif_export": true,
  "excluded_reports": [
    "text_wae_score_report",
    "text_wae_comparative_score_report"
  ],
  "exchange_parsing_rules": {
    "MY-CONTEST": {
      "regex": "(\\d{3})\\s+([A-Z]{2})\\s+([A-Z0-9/]+)\\s+(\\d{3})\\s+([A-Z]{2})",
      "groups": [
        "SentRST",
        "SentLocation",
        "Call",
        "RST",
        "RcvdLocation"
      ]
    }
  },
  "multiplier_rules": [
    {
      "name": "Sections",
      "value_column": "Mult1",
      "name_column": "Mult1Name",
      "totaling_method": "once_per_log"
    }
  ],
  "default_qso_columns": [
    "ContestName",
    "MyCall",
    "Frequency",
    "Mode",
    "Datetime",
    "SentRST",
    "SentLocation",
    "Call",
    "RST",
  ]
}
```

```

    "RcvdLocation",
    "Band",
    "Date",
    "Hour",
    "Dupe",
    "DXCCName",
    "DXCCPfx",
    "Run",
    "QSOPoints",
    "Mult1",
    "Mult1Name"
]
}

```

Reference Examples

For real-world examples, examine existing contest definitions in `contest_tools/contest_definitions/`:

- **Simple contests:** `cq_ww_cw.json`, `cq_wpx_cw.json`
- **Complex multipliers:** `arrl_dx_cw.json`, `naqp.json`
- **Custom parsers:** `arrl_ss.json`, `wae_cw.json`
- **Inheritance:** Check how `arrl_ss.json` is used by mode-specific variants
- **Special rules:** `wae_cw.json` (QTCs, custom scoring), `arrl_fd.json` (unique bands)

When Python Code is Needed

While most contests can be defined purely in JSON, you may need custom Python modules for:

1. **Custom Scoring:** Create a module in `contest_tools/contest_specific_annotations/` (e.g., `wae_scoring.py`)
2. **Custom Parsing:** Create a module in `contest_tools/contest_specific_annotations/` (e.g., `arrl_dx_parser.py`)
3. **Custom Multiplier Resolution:** Create a module in `contest_tools/contest_specific_annotations/` (e.g., `arrl_ss_multiplier_resolver.py`)
4. **Custom Location Resolution:** Create a module in `contest_tools/contest_specific_annotations/` (e.g., `wae_location_resolver.py`)
5. **Custom Dupe Checking:** Create a module in `contest_tools/contest_specific_annotations/` (e.g., `laqp_dupe_checker.py`)

When Custom Dupe Checking is Needed:

- Rover stations that can be worked from multiple locations (e.g., state QSO parties with county/parish rovers)
- Contests where dupe checking must consider exchange data (e.g., location codes in exchange)
- Asymmetric dupe rules (different rules for in-state vs out-of-state stations)

- Any contest where standard (Call, Band, Mode) dupe checking is insufficient

Function Signature:

```
def check_dupes(df: pd.DataFrame, my_location_type: Optional[str],
               root_input_dir: str, contest_def: ContestDefinition) -> pd.DataFrame:
    """
    Performs custom duplicate QSO checking.
    Returns a new DataFrame with the 'Dupe' column updated.

    Note: Standard dupe checking runs first during apply_annotations().
    Custom dupe checker runs after multiplier resolution (so location data
    is available) but before scoring. It can override the standard dupe flags.
    """
    # Implementation
    return df.copy() # Always return new DataFrame (immutability)
```

Execution Order:

1. Standard dupe checking (during `apply_annotations()`)
2. Multiplier resolution (populates location/exchange data)
3. **Custom dupe checking** (if specified, can override standard dupes)
4. Scoring (uses final Dupe column)

Reference existing modules in these directories for implementation patterns.

Data File Requirements

Some contests require external data files (.dat files) to be placed in the `data/` directory within your input directory. These files contain multiplier lists, section definitions, and other contest-specific data.

Required for All Contests:

- `cty.dat`: Country/prefix database (required for all contests)
- `band_allocations.dat`: Frequency validation data (required for all contests)

Contest-Specific Data Files:

Contest	Required Data File	Purpose
ARRL 10 Meter	<code>arrl_10_mults.dat</code>	Multiplier definitions
ARRL DX (CW/SSB)	<code>ARRLDXmults.dat</code>	Multiplier definitions
ARRL Sweepstakes	<code>SweepstakesSections.dat</code>	Section definitions
ARRL Field Day	<code>SweepstakesSections.dat</code>	Section definitions
CQ 160-Meter	<code>CQ160mults.dat</code>	Multiplier definitions
NAQP	<code>NAQPmults.dat</code>	Multiplier definitions
IARU HF	<code>iaru_officials.dat</code>	Official station list

How to Determine if Your Contest Needs Data Files:

1. **Check existing similar contests:** If your contest is similar to an existing one (e.g., uses ARRL sections), check what data files that contest uses.
2. **Check custom multiplier resolvers:** If your contest uses `custom_multiplier_resolver`, examine that Python module to see which data files it loads.
3. **Simple contests may not need data files:** Contests that use standard multipliers (e.g., CQ Zones, DXCC countries) typically don't require additional data files, as these are derived from the `cty.dat` file.

Where to Place Data Files:

- Data files must be in: `{CONTEST_INPUT_DIR}/data/`
- The system automatically looks in this location
- Ensure files are named exactly as expected (case-sensitive on some systems)

Exchange Parsing: Standard vs Custom

Understanding when to use standard parsing versus custom parsers is crucial for correctly adding a contest.

Standard Parsing (Most Contests):

The standard Cabrillo parser uses `exchange_parsing_rules` from your JSON definition to parse QSO exchanges. This works for most contests with straightforward exchange formats.

When Standard Parsing Works:

- Exchange format is consistent across all QSOs
- Exchange can be parsed with a single regex pattern
- No special handling needed for different operator locations or modes

Example: CQ WW, CQ WPX use standard parsing with simple regex patterns.

Custom Parsers (`custom_parser_module`):

Some contests need custom parsing logic when:

- Exchange format varies by operator location (e.g., W/VE vs DX in ARRL DX)
- Exchange format varies by mode (e.g., CW vs SSB in some contests)
- Special handling is needed (e.g., QTC lines in WAE contests)
- Pre-processing is required before parsing

Important: Even when using a custom parser, you still need `exchange_parsing_rules` in your JSON definition. The custom parser uses these rules to parse the exchange portion of each QSO line.

How the System Chooses:

1. If `custom_parser_module` is specified in JSON ¶ Custom parser is used
2. Otherwise ¶ Standard parser uses `exchange_parsing_rules` from JSON

Examples:

- `cq_ww_cw.json`: No `custom_parser_module` ¶ Uses standard parser
- `arrl_dx_cw.json`: Has `custom_parser_module: "arrl_dx_parser"` ¶ Uses custom parser (because W/VE and DX have different exchange formats)

Multiplier Resolution Flow

Understanding how multipliers are resolved helps you configure `multiplier_rules` correctly.

The Resolution Process:

1. **QSO Data Collection:** After parsing, each QSO has basic data (callsign, band, mode, exchange components, etc.)
2. **Location Determination:** The system determines the operator's location type (W/VE, DX, etc.) using:
 - `cty.dat` lookup of the operator's callsign
 - `custom_location_resolver` if specified
3. **Multiplier Resolution:** For each QSO, multipliers are resolved using one of these methods:

Method A: Direct Column Mapping (`source_column`)

```
{
  "name": "Zones",
  "source_column": "Zone",
  "value_column": "Mult2"
}
```

- Uses a column already present in the QSO data (e.g., Zone from exchange parsing)
- No additional lookup needed

Method B: Pre-defined Source (`source`)

```
{
  "name": "Countries",
  "source": "wae_dxcc",
  "value_column": "Mult1"
}
```

- Uses a built-in multiplier source (e.g., WAE DXCC list)

- System handles the lookup automatically

Method C: Custom Resolver (`custom_multiplier_resolver`)

```
{
  "custom_multiplier_resolver": "arrl_ss_multiplier_resolver"
}
```

- Python module performs custom logic
- Typically used when multipliers come from data files or need complex logic
- The resolver reads `multiplier_rules` from JSON to know which columns to populate

4. Multiplier Counting: After resolution, multipliers are counted according to `totaling_method`:

- `once_per_log`: Each unique multiplier counts once
- `sum_by_band`: Same multiplier on different bands counts separately
- `once_per_mode`: Count once per mode (for multi-mode contests)

Common Patterns:

- **Simple contests (CQ WW, CQ WPX)**: Use `source_column` pointing to Zone or rely on DXCC from `cty.dat`
- **Section-based contests (ARRL SS, ARRL FD)**: Use `custom_multiplier_resolver` with `SweepstakesSections.dat`
- **Complex contests (NAQP, ARRL DX)**: Use `custom_multiplier_resolver` with contest-specific data files

Field Dependencies

Some fields require or work together with other fields. Understanding these relationships prevents configuration errors.

Required Combinations:

If You Use	You Must Also Provide
<code>score_formula: "custom"</code>	<code>scoring_module</code> (Python module name)
<code>custom_parser_module</code>	<code>exchange_parsing_rules</code> (parser still uses these)
<code>custom_multiplier_resolver</code>	<code>multiplier_rules</code> (resolver reads these)
<code>time_series_calculator</code>	Usually <code>scoring_module</code> (for custom scoring)
<code>applies_to</code> in <code>multiplier_rules</code>	Location-specific logic (W/VE vs DX)

Optional but Recommended:

- If using `multiplier_rules` with `source_column`, ensure that column exists in your exchange parsing (e.g., if `source_column: "Zone"`, parse Zone in `exchange_parsing_rules`)

- If using `custom_multiplier_resolver`, check which data files it requires and ensure they exist
- If using `excluded_reports`, verify the report names match actual report modules

Validation Tips:

- The system will raise errors if required combinations are missing
- Missing `scoring_module` when `score_formula: "custom"` will result in zero points
- Missing `exchange_parsing_rules` when using standard parser will fail to parse QSOs

Testing Your Contest Definition

After creating your contest definition, test it thoroughly before using it in production.

Step 1: Validate JSON Syntax

Use a JSON validator or Python to check syntax

```
python -m json.tool contest_tools/contest_definitions/my_contest.json
```

Step 2: Test with a Sample Log

1. **Prepare a test log:** Use a small Cabrillo log file from the contest you're adding
2. **Run a simple analysis:** Use the web interface to upload and analyze test logs, or use the Django test client for automated testing.
3. **Check for errors:** Look for:
 - `FileNotFoundError`: Contest definition not found (check filename and `contest_name`)
 - `ValueError`: Parsing errors, missing fields, or configuration issues
 - Warnings about missing multipliers or zero points

Step 3: Verify Parsing

Check that QSOs are parsed correctly:

- Open the generated CSV or check the summary report
- Verify exchange components are in correct columns
- Check that multipliers are being identified (if applicable)

Step 4: Verify Scoring

- Check that QSO points are calculated correctly
- Verify multiplier counts match expected values
- Compare against known good scores if available

Step 5: Test Reports

Generate different report types to ensure they work: Use the web interface to upload logs and verify all report types are generated correctly, or use the Django test client for automated testing.

Common Test Scenarios:

- **Single QSO test:** Create a minimal log with one QSO to verify basic parsing
- **Multi-band test:** Ensure multipliers work across bands correctly
- **Edge cases:** Test with unusual callsigns, portable indicators, etc.

Common Pitfalls and Solutions

Learn from common mistakes to avoid frustration.

Pitfall 1: Contest Name Mismatch

Problem: `FileNotFoundError: No definition file found for 'MY-CONTEST'`

Causes:

- Filename doesn't match (e.g., created `my_contest.json` but contest name is `MY-CONTEST-CW`)
- `contest_name` in JSON doesn't match `CONTEST:` header in Cabrillo file

Solution:

- Ensure filename follows naming rules: `contest_name.lower().replace('-', '_').replace(' ', '_') + '.json'`
- Ensure `contest_name` in JSON exactly matches the `CONTEST:` header (case-sensitive)

Pitfall 2: Exchange Parsing Fails

Problem: QSOs parsed but exchange fields are empty or incorrect

Causes:

- Regex pattern doesn't match actual exchange format
- Number of capturing groups doesn't match `groups` array length
- Whitespace handling (tabs vs spaces)

Solution:

- Examine actual QSO lines from a Cabrillo file
- Test regex pattern separately (use online regex tester)
- Use `\s+` for flexible whitespace matching
- Ensure `groups` array has same number of elements as capturing groups in regex

Pitfall 3: Multipliers Not Counting

Problem: Multipliers identified but count is zero or incorrect

Causes:

- `totaling_method` incorrect for contest rules
- `custom_multiplier_resolver` not working correctly
- Missing data files
- `value_column` and `name_column` not matching what resolver populates

Solution:

- Check contest rules for how multipliers should be counted
- Verify data files exist if using custom resolver
- Check resolver code to see which columns it populates
- Ensure `multiplier_rules` column names match resolver output

Pitfall 4: Zero Points for All QSOs

Problem: All QSOs show 0 points

Causes:

- `scoring_module` missing when `score_formula: "custom"`
- Scoring module has errors
- `scoring_module` not found (import error)

Solution:

- If using `score_formula: "custom"`, ensure `scoring_module` is specified
- Check Python module exists and is importable
- Review error logs for import or execution errors
- For standard formulas, ensure `score_formula` is set correctly

Pitfall 5: Inheritance Not Working

Problem: Inherited fields not appearing or overrides not taking effect

Causes:

- Parent file not found
- Field name typo in `inherits_from`
- Deep merge not working as expected

Solution:

- Verify parent file exists and is readable
- Check `inherits_from` value matches parent filename (without `.json`)
- Remember: child values override parent values (deep merge)

Pitfall 6: Custom Parser Not Used

Problem: Custom parser specified but standard parser runs instead

Causes:

- Module not found (import error)
- Module doesn't have required function signature

- Module path incorrect

Solution:

- Ensure module is in `contest_tools/contest_specific_annotations/`
- Verify module has `parse_log()` function with correct signature
- Check module name matches `custom_parser_module` value exactly

Sweepstakes-Specific Implementation

ARRL Sweepstakes has unique scoring rules that require specialized handling:

Key Characteristics:

- **Multiplier Counting:** Uses `totaling_method`: "once_per_log" - each section counts once per contest, not per band
- **Fixed Multiplier Maximum:** There are exactly 85 possible sections (loaded from `SweepstakesSections.dat`)
- **Enhanced Missed Multipliers Report:** Specialized report (`text_enhanced_missed_multipliers.py`) provides detailed breakdown

Implementation Details:

1. Multiplier Dashboard Features:

- Fixed scale progress bars with reference line at 85 multipliers
- Band Spectrum automatically suppressed (multipliers are contest-wide, not per-band)
- Enhanced Missed Multipliers report appears in dashboard when missed multipliers exist

2. Enhanced Missed Multipliers Report:

- Report ID: `enhanced_missed_multipliers`
- Only generates for contests with `contest_name.startswith("ARRL-SS")`
- Uses `MultiplierStatsAggregator.get_missed_data(mult_name, mode_filter, enhanced=True)`
- Returns detailed breakdown with:
 - Which logs worked each missed multiplier
 - Bands/modes where worked
 - Aggregated Run/S&P/Unknown counts across all bands/modes

3. Data Aggregation:

- `MultiplierStatsAggregator._compute_enhanced_breakdown()` processes QSO data to extract:
 - Worked-by callsigns (from multiplier sets)
 - Band/mode distribution (from QSO records)
 - Run/S&P/Unknown classification (from `Run` column in dataframe)

4. Report Generation:

- Report must be added to `is_multiplier_report` list in `report_generator.py`
- Generates with session-level filename: `enhanced_missed_multipliers--{callsigns}.txt`
- Only generates when missed multipliers exist (skips if none found)

Files Involved:

- `contest_tools/data_aggregators/multiplier_stats.py`: Enhanced breakdown computation
- `contest_tools/reports/text_enhanced_missed_multipliers.py`: Report generation
- `web_app/analyzer/views.py`: Dashboard discovery and display logic
- `web_app/analyzer/templates/analyzer/multiplier_dashboard.html`: UI integration

Troubleshooting

When things go wrong, use this systematic approach to diagnose and fix issues.

Error: "No definition file found for 'CONTEST-NAME'"

1. Check filename matches naming convention
2. Verify file is in `contest_tools/contest_definitions/`
3. Check `contest_name` in JSON matches Cabrillo header exactly
4. Try implicit inheritance (remove last hyphen segment from contest name)

Error: "Parsing rule 'CONTEST-NAME' not found"

1. Verify `exchange_parsing_rules` has key matching contest name
2. Check if custom parser needs different key format (e.g., `CONTEST-NAME-W/VE`)
3. Ensure regex pattern is valid JSON (escape backslashes: `\\d` not `\d`)

Error: "Could not load scoring module"

1. Verify `scoring_module` name is correct
2. Check module exists in `contest_tools/scoring/` or `contest_tools/contest_specific_annotations/`
3. Ensure module has `calculate_points()` function
4. Check for Python syntax errors in module

Warning: "Could not determine own location"

1. Verify `cty.dat` file exists and is readable
2. Check callsign in Cabrillo header is valid
3. If using `custom_location_resolver`, verify it's working correctly

Multipliers Not Appearing:

1. Check if `custom_multiplier_resolver` is specified and working
2. Verify required data files exist
3. Check `multiplier_rules` column names match what resolver populates

4. Ensure QSOs have necessary data (callsign, location, etc.) for multiplier resolution

Reports Not Generating:

1. Check if contest is in `excluded_reports` list
2. Verify report modules exist
3. Check for errors in report generation logs
4. Ensure required data is present (QSOs, multipliers, etc.)

Getting Help:

- Review existing contest definitions for similar contests
 - Check error messages carefully—they often point to the exact issue
 - Test with minimal configuration first, then add complexity
 - Use the reference examples listed earlier in this guide
-

6. Testing

Web Regression Testing

The project includes automated regression testing for the web version using Django's test client. Tests verify that report generation produces consistent output across releases.

Test Infrastructure:

- **Location:** `test_code/web_regression_test.py`
- **Test Data:** `regression_baselines/Logs/` (version-controlled test logs)
- **Baselines:** `regression_baselines/web_baseline/{version_tag}/{date_time}/` (versioned ZIP archives organized by version and test run time)

Running Tests:

Create a New Baseline:

```
regression_baselines\run_tests.bat --create-baseline v1.0.0-alpha.10
```

This generates baseline ZIP archives for all test cases and stores them in `regression_baselines/web_baseline/v1.0.0-alpha.10/{date_time}/`, where `{date_time}` is automatically created based on the test run start time (format: YYYY-MM-DD HHMM, e.g., 2026-01-22 0102).

Run Regression Tests:

```
regression_baselines\run_tests.bat --baseline v1.0.0-alpha.10
```

This compares current output against the specified baseline and reports any differences.

Requirements:

- Conda environment `cla` with Django installed
- Test logs in `regression_baselines/Logs/`
- Project root as current directory

How It Works:

1. Tests use Django test client (in-process, no HTTP server needed)
2. Uploads test logs via Django test client
3. Downloads generated ZIP archives
4. Compares against baseline archives (sanitizing dynamic content like UUIDs, timestamps)
5. Reports differences for investigation

Test Cases:

- 18 test cases covering multiple contests (CQ WW, ARRL SS, CQ 160, CQ WPX, ARRL 10, ARRL DX, IARU HF, WAE, NAQP)
- Single-log and multi-log scenarios
- Different contest types (symmetric, asymmetric, custom calculators)

For More Information:

- `regression_baselines/README.md` - Test data repository documentation
- `DevNotes/REGRESSION_TESTING_WEB_VERSION_STRATEGY.md` - Detailed testing strategy

JSON Quick Reference

Key	Type	Required	Description
<code>contest_name</code>	string	Yes	Matches the <code>CONTEST:</code> Cabrillo header exactly
<code>dupe_check_scope</code>	string	Yes	" <code>per_band</code> " or " <code>all_bands</code> "
<code>score_formula</code>	string	Yes	" <code>points_times_mults</code> ", " <code>qsos_times_mults</code> ", " <code>total</code> "
<code>valid_bands</code>	array	Yes	Array of band names (e.g., ["80M", "40M", "20M"])
<code>exchange_parsing_rules</code>	object	Usually	Regex patterns for parsing QSO exchanges
<code>multiplier_rules</code>	array	Usually	Array of multiplier definition objects
<code>default_qso_columns</code>	array	No	Columns to include in reports
<code>scoring_module</code>	string	No	Python module for custom scoring
<code>custom_parser_module</code>	string	No	Python module for custom parsing
<code>custom_multiplier_resolver</code>	string	No	Python module for custom multiplier logic
<code>custom_location_resolver</code>	string	No	Python module for custom location logic
<code>custom_dupe_checker</code>	string	No	Python module for custom duplicate checking
<code>custom_adif_exporter</code>	string	No	Python module for custom ADIF export
<code>time_series_calculator</code>	string	No	Python module for custom time series
<code>inherits_from</code>	string	No	Parent contest definition to inherit from
<code>excluded_reports</code>	array	No	Report names to skip
<code>included_reports</code>	array	No	Report names to include (if specified, only these)

Key	Type	Required	Description
contest_period	object	No	Contest start/end day and time
operating_time_rules	object	No	Operating time restrictions
mults_from_zero_point_qsos	boolean	No	Whether zero-point QSOs yield multipliers (default: t
mutually_exclusive_mults	array	No	Groups of mutually exclusive multipliers
multiplier_report_scope	string	No	"per_band" or "all_bands"
enable_adif_export	boolean	No	Enable ADIF export functionality
points_header_label	string	No	Custom label for points column
is_naqp_ruleset	boolean	No	Special flag for NAQP rules